**Міністерство освіти і науки України**
**Національний технічний університет України**
**«Київський політехнічний інститут імені Ігоря Сікорського»**
**Факультет інформатики та обчислювальної техніки**
**Кафедра обчислювальної техніки**

# Лабораторна робота№5

з дисципліни
«Об’єктно орієнтоване програмування»
на тему
“Знайомство із середовищем розробки програм Microsoft Visual Studio та складання модульних проєктів програм на C++”

Виконав:
Студент групи ІМ-33
Набока Тимур Максимович
номер у списку групи: 17

Перевірив:
Порєв В.М.

Київ 2024

**Мета:**

Мета роботи – отримати вміння та навички проектування класів, виконавши модернізацію коду графічного редактора в об’єктно-орієнтованому стилі для забезпечення зручного додавання нових типів об'єктів.

**Завдання**

1. Створити у середовищі MS Visual Studio C++ проект Win32 з ім’ям **Lab4**.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. Налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.
6. Оформити звіт.

## mainF.py

```
import pygame
from ShapeEditor import ShapeEditor
from MyTable import MyTable


field = 600
screen = pygame.display.set_mode((field, field))
clock = pygame.time.Clock()
pygame.display.set_caption('copy_of_paint')
screen.fill('white')
table_sam = MyTable()
table_sam.screen = screen

# print('table_sam->1', table_sam)
# table_sam2 = MyTable()
# print('table_sam->2', table_sam2)

shape_editor = ShapeEditor(field, screen, clock, table_sam)

shape_editor.code_runner()
```

## ShapeEditor.py

```python
import pygame
from Menu import Menu
from LineEditor import LineEditor
from PointEditor import PointEditor
from RectEditor import RectEditor
from ElipsEditor import ElipsEditor
from DumbbellEditor import DumbbellEditor
from CubeEditor import CubeEditor
from ShapeChooser import ShapeChooser
from MyTable import MyTable

class ShapeEditor:
    def __init__(self, field, screen, clock, table_sam):
        self.table_sam = table_sam
        self.field = field
        self.screen = screen
        self.clock = clock
        self.ghost_table_active = False
        self.project_running = True

    def code_runner(self):
        menu = Menu(self.field, self.screen)
        shape_list, shape_list_name, rgb_colors, table =
menu.createMenu()
        line_editor = LineEditor(self.screen)
        point_editor = PointEditor(self.screen)
        rect_editor = RectEditor(self.screen)
        elips_editor = ElipsEditor(self.screen)
        shape_chooser = ShapeChooser(self.screen, shape_list,
shape_list_name)
        dumbbell_editor = DumbbellEditor(self.screen)
        cube_editor = CubeEditor(self.screen)

        chosen_shape = None
        active_color = (0, 0, 0)  # Default color is black

        editor_list = {
            'point': point_editor,
            'line': line_editor,
            'rectangle': rect_editor,
            'circle': elips_editor,
            'dumbbell': dumbbell_editor,
            'cube': cube_editor,
        }

        settings_obj = {
            'shape': 'shape',
            'start': (0, 0),
            'end': (0, 0),
            'screen': self.screen,
            'color': active_color
        }
```

```
        while self.project_running:
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    self.project_running = False

                menu.createMenu()

                if event.type == pygame.MOUSEWHEEL:
                    self.table_sam.handle_scroll(event)

                if event.type == pygame.MOUSEBUTTONDOWN:
                    if event.pos[1] < 70 and event.button == 1:

                        if table.collidepoint(event.pos):
                            self.ghost_table_active =
self.table_sam.handle_events()
                            self.table_sam.draw_table()
                            print('action happens
============================')



                        for color_rect in rgb_colors:
                            if color_rect.collidepoint(event.pos):
                                active_color =
self.screen.get_at(event.pos)
                                settings_obj['color'] =
active_color

menu.choose_color_for_detector(active_color)
                                print(f"Selected color:
{active_color}")
                                break
                        else:
                            chosen_shape =
shape_chooser.choose(event)
                            settings_obj['shape'] = chosen_shape
                            print(f"Chosen shape: {chosen_shape}")

                    elif chosen_shape is not None:
                        if chosen_shape == 'dumbbell':

editor_list[chosen_shape].on_mouse_down(event, active_color)
                        else:

editor_list[chosen_shape].on_mouse_down(event)

                        settings_obj['start'] = event.pos
                        editor_list[chosen_shape].start_dragging()
```

```
                if event.type == pygame.MOUSEMOTION and
chosen_shape is not None:
                    editor_list[chosen_shape].on_mouse_move(event)
                
                if event.type == pygame.MOUSEBUTTONUP and
chosen_shape is not None:
                    if event.button == 1:
                        editor_list[chosen_shape].stop_dragging()

editor_list[chosen_shape].on_mouse_up(event)
                        settings_obj['end'] = event.pos

editor_list[chosen_shape].draw(active_color)
                        
                        if event.pos[1] > 70:
                            self.table_sam.store_figure([
                             settings_obj['shape'],
                             settings_obj['start'],
                             settings_obj['end']])
                            
                            self.table_sam.draw_table()
                        

            pygame.display.flip()
            self.clock.tick(60)

        pygame.quit()
```

## ShpaeChooser.py

```
import pygame

class ShapeChooser:
    def __init__(self, screen, shape_list, shape_list_name):
        self.screen = screen
        self.shape_list = shape_list
        self.shape_list_name = shape_list_name
        self.active_shape = 0

    def choose(self, event):
        for i in range(len(self.shape_list)):
            if self.shape_list[i].collidepoint(event.pos):
                self.active_shape = self.screen.get_at(event.pos)
                return self.shape_list_name[i]
```

## Menu.py

```
import pygame
pygame.init()


class Menu:
    def __init__(self, field, screen):
        self.field = field
        self.screen = screen
        self.currect_color = (255, 255, 255)


    def choose_color_for_detector(self, color):
        self.currect_color = color
        print("detector:", self.currect_color)

    def createMenu(self):
        menu_height = (self.field*12)/100
        brash_block_size = round(menu_height/1.4)
        pygame.draw.rect(self.screen, 'gray', [0, 0, self.field,
menu_height])
        pygame.draw.line(self.screen, 'black', (0,menu_height),
(self.field, menu_height), 3)

        rectangale = pygame.draw.rect(self.screen, 'black',
[10,10,brash_block_size,brash_block_size])
        pygame.draw.rect(self.screen, 'white', [15, 20, 40, 30])

        circle = pygame.draw.rect(self.screen, 'black',
[70,10,brash_block_size,brash_block_size])
        pygame.draw.circle(self.screen, 'white', (95, 35), 20)
        pygame.draw.circle(self.screen, 'black', (95, 35), 18)


        point = pygame.draw.rect(self.screen, 'black',
[130,10,brash_block_size,brash_block_size])
        pygame.draw.circle(self.screen, 'white', (155, 35), 10)

        line = pygame.draw.rect(self.screen, 'black',
[190,10,brash_block_size,brash_block_size])
        pygame.draw.line(self.screen, 'white', (190, 10), (240,
60) , 3)

        dumbbell = pygame.draw.rect(self.screen, 'black',
[250,10,brash_block_size,brash_block_size])
        pygame.draw.circle(self.screen, 'white', (265, 45), 8)
        pygame.draw.line(self.screen, 'white', (265, 45), (290,
25) , 3)
        pygame.draw.circle(self.screen, 'white', (290, 25), 8)

        cube = pygame.draw.rect(self.screen, 'black',
[310,10,brash_block_size,brash_block_size])
        pygame.draw.rect(self.screen, 'white', [330, 20, 25, 25])
```

```
        pygame.draw.rect(self.screen, 'black', [332, 22, 20, 20])
        pygame.draw.line(self.screen, 'white', (330, 20), (315,
30), 3)
        pygame.draw.line(self.screen, 'white', (352, 20), (340,
30), 3)
        pygame.draw.line(self.screen, 'white', (352, 45), (340,
52), 3)
        pygame.draw.rect(self.screen, 'white', [315, 30, 25, 25])
        pygame.draw.rect(self.screen, 'black', [317, 32, 20, 20])



        shape_list = [rectangale, circle, point, line, dumbbell,
cube]
        shape_list_name = ['rectangle', 'circle', 'point', 'line',
'dumbbell', 'cube']

        pygame.draw.circle(self.screen, self.currect_color, (400,
35), 30)
        pygame.draw.circle(self.screen, 'dark grey', (400, 35),
30, 3)


        table = pygame.draw.rect(self.screen, 'black', [440,
10,brash_block_size,brash_block_size])
        pygame.draw.rect(self.screen, 'white', [445,
17,brash_block_size-10, brash_block_size-15])
        pygame.draw.rect(self.screen, 'black', [446,
18,brash_block_size-13, brash_block_size-18])
        pygame.draw.line(self.screen, 'white', (445, 25), (485,
25), 3)
        pygame.draw.line(self.screen, 'white', (465, 17), (465,
50), 2)


        red = pygame.draw.rect(self.screen, 'red', [self.field -
35, 22, 25, 25])
        blue = pygame.draw.rect(self.screen, 'blue', [self.field -
60, 22, 25, 25])
        green = pygame.draw.rect(self.screen, 'green', [self.field
- 85, 22, 25, 25])



        rgb_colors = [red, blue, green]

        return shape_list, shape_list_name, rgb_colors, table


```

## Shape.py

```
from abc import ABC, abstractmethod

class Shape(ABC):

    @abstractmethod
    def detect_start():
        pass

    @abstractmethod
    def detect_end():
        pass

    @abstractmethod
    def draw():
        pass
```

## Editor.py

```
from abc import ABC, abstractmethod

class Editor(ABC):

    @abstractmethod
    def on_mouse_down():
        pass

    @abstractmethod
    def on_mouse_up():
        pass

    @abstractmethod
    def on_mouse_move():
        pass
```

## PointShape.py

```
from Shape import Shape
import pygame

class PointShape(Shape):
    def __init__(self, screen):
        self.screen = screen
        self.start_x = None
        self.start_y = None
        self.end_x = None
        self.end_y = None
```

```
    def detect_start(self, coords):
        self.start_x = coords[0]
        self.start_y = coords[1]

    def detect_end(self, coords):
        self.end_x = coords[0]
        self.end_y = coords[1]

    def update_elment(self, temp_coords):
        pass

    def draw(self, color):
        if self.start_x is not None and self.start_y is not None:
            if self.start_y > 70 and self.end_y > 70:
                element = pygame.draw.circle(self.screen, color,
(self.start_x, self.start_y), 10)
                return element
```

## PointEditor.py

```
from Editor import Editor
import pygame
from PointShape import PointShape

class PointEditor(Editor):
    def __init__(self, screen):
        self.screen = screen
        self.point_shape_sam = PointShape(self.screen)

    def on_mouse_down(self, event):
        self.point_shape_sam.detect_start(event.pos)

    def on_mouse_up(self, event):
        self.point_shape_sam.detect_end(event.pos)
        # self.line_shape_sam.draw()

    def on_mouse_move(self, event):
        pass

    def start_dragging(self):
        pass

    def stop_dragging(self):
        pass

    def draw(self, color):
        self.point_shape_sam.draw(color)
```

## LineShape.py

```
import pygame
from Shape import Shape

class LineShape(Shape):
    def __init__(self, screen):
        self.screen = screen
        self.start_x = None
        self.start_y = None
        self.end_x = None
        self.end_y = None
        self.prev_coords = None  # Зберігаємо попередні координати
для стирання

    def detect_start(self, coords):
        self.start_x, self.start_y = coords
        self.prev_coords = None  # Скидаємо попередні координати
при новому старті

    def detect_end(self, coords):
        self.end_x, self.end_y = coords
        self.prev_coords = None  # Скидаємо попередні координати
при завершенні

    def update_elment(self, temp_coords):
        if self.start_y is not None and self.start_y > 70:
            # Стерти попередню тимчасову лінію кольором фону, якщо
вона існує
            if self.prev_coords:
                pygame.draw.line(self.screen, (255, 255, 255),
(self.start_x, self.start_y), self.prev_coords, 2)
            # Намалювати нову тимчасову лінію чорним кольором
            pygame.draw.line(self.screen, (0, 0, 0),
(self.start_x, self.start_y), temp_coords, 2)
            # Оновити координати попередньої лінії
            self.prev_coords = temp_coords

    def draw(self, color):
        if self.start_x is not None and self.start_y is not None
and self.end_y > 70:
            # Фінальна лінія, малюється обраним кольором
            pygame.draw.line(self.screen, color, (self.start_x,
self.start_y), (self.end_x, self.end_y), 2)
```

## LineEditor.py

```
from Editor import Editor
import pygame
```

```
from LineShape import LineShape

class LineEditor(Editor):
    def __init__(self, screen):
        self.screen = screen
        self.line_shape_sam = LineShape(self.screen)
        self.temp_coord = None
        # self.temp_coord_y = None
        self.drawing = None

    def on_mouse_down(self, event):
        self.line_shape_sam.detect_start(event.pos)

    def on_mouse_up(self, event):
        self.line_shape_sam.detect_end(event.pos)

    def start_dragging(self):
        self.drawing = True

    def stop_dragging(self):
        self.drawing = False

    def on_mouse_move(self, event):
        if event.pos[1] > 70 and self.drawing:
            self.temp_coord =
self.line_shape_sam.update_elment((event.pos))

    def draw(self, color):
        self.line_shape_sam.draw(color)
```

## ElipsShape.py

```
from Shape import Shape
import pygame

class ElipsShape(Shape):
    def __init__(self, screen):
        self.screen = screen
        self.start_x = None
        self.start_y = None
        self.end_x = None
        self.end_y = None
        self.prev_ellipse_rect = None  # Зберігаємо попередній
еліпс для стирання

    def detect_start(self, coords):
        self.start_x, self.start_y = coords
        self.prev_ellipse_rect = None  # Скидаємо попередній еліпс
при новому старті
```

```
    def detect_end(self, coords):
        self.end_x, self.end_y = coords
        self.prev_ellipse_rect = None  # Скидаємо попередній еліпс при завершенні

    def update_elment(self, temp_coords):
        if self.start_y is not None and self.start_y > 70:
            # Вираховуємо ширину і висоту тимчасового еліпса
            width = abs(temp_coords[0] - self.start_x)
            height = abs(temp_coords[1] - self.start_y)
            ellipse_x = min(self.start_x, temp_coords[0])
            ellipse_y = min(self.start_y, temp_coords[1])
            current_ellipse_rect = (ellipse_x, ellipse_y, width, height)

            # Стерти попередній тимчасовий еліпс, якщо він існує
            if self.prev_ellipse_rect:
                pygame.draw.ellipse(self.screen, (255, 255, 255), self.prev_ellipse_rect, 2)

            # Намалювати новий тимчасовий еліпс чорним кольором
            pygame.draw.ellipse(self.screen, (0, 0, 0), current_ellipse_rect, 2)

            # Оновити координати попереднього тимчасового еліпса
            self.prev_ellipse_rect = current_ellipse_rect

    def draw(self, color, filled=True):
        if None not in (self.start_x, self.start_y, self.end_x, self.end_y):
            if self.start_y > 70 and self.end_y > 70:
                width = abs(self.end_x - self.start_x)
                height = abs(self.end_y - self.start_y)
                ellipse_x = min(self.start_x, self.end_x)
                ellipse_y = min(self.start_y, self.end_y)

                if filled:
                    element = pygame.draw.ellipse(self.screen, color, (ellipse_x, ellipse_y, width, height))
                else:
                    element = pygame.draw.ellipse(self.screen, color, (ellipse_x, ellipse_y, width, height), 2)

                return element
```

## ElipsEditor.py

```
from Editor import Editor
import pygame
from ElipsShape import ElipsShape
```

```
class ElipsEditor(Editor):
    def __init__(self, screen):
        self.screen = screen
        self.elips_shape_sam = ElipsShape(self.screen)
        self.temp_coord = None
        self.drawing = None

    def on_mouse_down(self, event):
        self.elips_shape_sam.detect_start(event.pos)

    def on_mouse_up(self, event):
        self.elips_shape_sam.detect_end(event.pos)

    def start_dragging(self):
        self.drawing = True

    def stop_dragging(self):
        self.drawing = False

    def on_mouse_move(self, event):
        if event.pos[1] > 70 and self.drawing:
            self.temp_coord =
self.elips_shape_sam.update_elment((event.pos))

    def draw(self, color):
        self.elips_shape_sam.draw(color)
```

## RectShape.py

```
import pygame
from Shape import Shape


class RectShape(Shape):
    def __init__(self, screen):
        self.screen = screen
        self.start_x = None
        self.start_y = None
        self.end_x = None
        self.end_y = None
        self.prev_rect = None  # Зберігаємо попередній прямокутник
для стирання

    def detect_start(self, coords):
        self.start_x, self.start_y = coords
        self.prev_rect = None  # Скидаємо попередній прямокутник
при новому старті

    def detect_end(self, coords):
        self.end_x, self.end_y = coords
```

```
        self.prev_rect = None  # Скидаємо попередній прямокутник при завершенні

    def update_elment(self, temp_coords):
        if self.start_y is not None and self.start_y > 70:
            # Вираховуємо ширину і висоту тимчасового прямокутника
            width = abs(temp_coords[0] - self.start_x)
            height = abs(temp_coords[1] - self.start_y)
            rect_x = min(self.start_x, temp_coords[0])
            rect_y = min(self.start_y, temp_coords[1])
            current_rect = (rect_x, rect_y, width, height)

            # Стерти попередній тимчасовий прямокутник, якщо він існує
            if self.prev_rect:
                pygame.draw.rect(self.screen, (255, 255, 255), self.prev_rect, 2)

            # Намалювати новий тимчасовий прямокутник чорним кольором
            pygame.draw.rect(self.screen, (0, 0, 0), current_rect, 2)

            # Оновити координати попереднього тимчасового прямокутника
            self.prev_rect = current_rect

    def draw(self, color, filled=True):
        if None not in (self.start_x, self.start_y, self.end_x, self.end_y):
            if self.start_y > 70 and self.end_y > 70:
                width = abs(self.end_x - self.start_x)
                height = abs(self.end_y - self.start_y)
                rect_x = min(self.start_x, self.end_x)
                rect_y = min(self.start_y, self.end_y)

                if filled:
                    element = pygame.draw.rect(self.screen, color, (rect_x, rect_y, width, height))
                else:
                    element = pygame.draw.rect(self.screen, color, (rect_x, rect_y, width, height), 2)

                return element
```

## RectEditor.py

```
from Editor import Editor
import pygame
from RectShape import RectShape
```

```
class RectEditor(Editor):
    def __init__(self, screen):
        self.screen = screen
        self.rect_shape_sam = RectShape(self.screen)
        self.temp_coord = None
        # self.temp_coord_y = None
        self.drawing = None

    def on_mouse_down(self, event):
        self.rect_shape_sam.detect_start(event.pos)

    def on_mouse_up(self, event):
        self.rect_shape_sam.detect_end(event.pos)
        # self.line_shape_sam.draw()

    def on_mouse_up(self, event):
            self.rect_shape_sam.detect_end(event.pos)

    def start_dragging(self):
        self.drawing = True

    def stop_dragging(self):
        self.drawing = False

    def on_mouse_move(self, event):
        if event.pos[1] > 70 and self.drawing:
            self.temp_coord =
self.rect_shape_sam.update_elment((event.pos))

    def draw(self, color):
        self.rect_shape_sam.draw(color)
```

## DumbbellEditor.py

```
import pygame
from PointEditor import PointEditor
from LineEditor import LineEditor
from DumbbellShape import DumbbellShape

class DumbbellEditor(PointEditor, LineEditor):
    def __init__(self, screen):
        super().__init__(screen)
        self.screen = screen
        self.dumbbell_shape_sam = DumbbellShape(self.screen)

    def on_mouse_down(self, event, color):
        self.dumbbell_shape_sam.detect_start(event.pos, color)

    def on_mouse_up(self, event):
        self.dumbbell_shape_sam.detect_end(event.pos)
```

```
    def start_dragging(self):
        self.drawing = True

    def stop_dragging(self):
        self.drawing = False

    def on_mouse_move(self, event):
        if event.pos[1] > 70 and self.drawing:
            self.temp_coord =
self.dumbbell_shape_sam.update_elment((event.pos))

    def draw(self, color):
        self.dumbbell_shape_sam.draw(color)
```

## DumbbellShape.py

```
import pygame
from PointShape import PointShape
from LineShape import LineShape

class DumbbellShape(PointShape, LineShape):
    def __init__(self, screen):
        super().__init__(screen)

    def detect_start(self, coords, color):
        LineShape.detect_start(self, coords)
        PointShape.detect_start(self, coords)
        PointShape.draw(self, color)

    def detect_end(self, coords):
        LineShape.detect_end(self, coords)
        PointShape.detect_start(self, coords)

    def update_elment(self, temp_coords):
        LineShape.update_elment(self, temp_coords)

    def draw(self, color):
        if self.start_x is not None and self.start_y is not None
and self.end_y > 70:
            PointShape.draw(self, color)
            LineShape.draw(self, color)
```

## CubeEditor.py

```
import pygame
```

```python
from RectEditor import RectEditor
from LineEditor import LineEditor
from CubeShape import CubeShape


class CubeEditor(RectEditor, LineEditor):
    def __init__(self, screen):
        super().__init__(screen)
        self.screen = screen
        self.cube_shape_sam = CubeShape(self.screen)
        self.temp_coord = None
        self.temp_coord_y = None
        self.drawing = None

    # def on_mouse_down(self, event):
    #     self.cube_shape_sam.detect_start(event.pos)

    def on_mouse_down(self, event):
        # x = event.pos[0]
        # y = event.pos[1]

        # xPdx = event.pos[0] + 50
        # yPdx = event.pos[1] + 50

        # self.cube_shape_sam.detect_start([x, y, xPdx, yPdx])
        self.cube_shape_sam.detect_start(event.pos)

    def on_mouse_up(self, event):
        # x = event.pos[0]
        # y = event.pos[1]

        # xPdx = event.pos[0] + 50
        # yPdx = event.pos[1] + 50
        # self.cube_shape_sam.detect_end([x, y, xPdx, yPdx])
        self.cube_shape_sam.detect_end(event.pos)

    def start_dragging(self):
        self.drawing = True

    def stop_dragging(self):
        self.drawing = False

    def on_mouse_move(self, event):
        if event.pos[1] > 70 and self.drawing:
            x = event.pos[0]
            y = event.pos[1]

            xPdx = event.pos[0] + 50
            yPdx = event.pos[1] + 50
            self.cube_shape_sam.update_elment([x, y, xPdx, yPdx])
            # self.cube_shape_sam.update_elment(event.pos)
```

```
    def draw(self, color):
        self.cube_shape_sam.draw(color)
```

## CubeShape.py

```
import pygame
from RectShape import RectShape
from LineShape import LineShape


class CubeShape(RectShape, LineShape):
    def __init__(self, screen):
        super().__init__(screen)
        self.screen = screen
        self.start_x = None
        self.start_y = None
        self.end_x = None
        self.end_y = None
        self.prev_cube_back = None
        self.prev_cube_front = None


    def detect_start(self, coords):
        self.start_x, self.start_y = coords
        # self.prev_rect = None  # Скидаємо попередній прямокутник
при новому старті
        # RectShape.detect_start(self, (coords[0], coords[1]))
        # RectShape.detect_start(self, (coords[2], coords[3]))


    def detect_end(self, coords):
        self.end_x, self.end_y = coords
        # self.prev_rect = None  # Скидаємо попередній прямокутник
при завершенні
        # RectShape.detect_end(self, (coords[0], coords[1]))
        # RectShape.detect_end(self, (coords[2], coords[3]))


    def update_elment(self, temp_coords):
        # Calculate the front face of the cube (based on start and
temporary coordinates)
        rect_x_f = min(self.start_x, temp_coords[0])
        rect_y_f = min(self.start_y, temp_coords[1])
        width_f = abs(temp_coords[0] - self.start_x)
        height_f = abs(temp_coords[1] - self.start_y)
        current_cube_front = (rect_x_f, rect_y_f, width_f,
height_f)

        # Calculate the back face of the cube (offset from the
front face)
```

```
        offset = 50  # Cube depth
        rect_x_b = rect_x_f + offset
        rect_y_b = rect_y_f + offset
        current_cube_back = (rect_x_b, rect_y_b, width_f,
height_f)

        # Clear the previous cube (if it exists) by drawing white
rectangles
        if self.prev_cube_back and self.prev_cube_front:
            pygame.draw.rect(self.screen, (255, 255, 255),
self.prev_cube_back, 2)
            pygame.draw.rect(self.screen, (255, 255, 255),
self.prev_cube_front, 2)
            self._clear_connection_lines()

        # Draw the new cube
        pygame.draw.rect(self.screen, (0, 0, 0),
current_cube_back, 2)
        pygame.draw.rect(self.screen, (0, 0, 0),
current_cube_front, 2)
        self._draw_connection_lines(current_cube_front,
current_cube_back)

        # Save the current cube as the previous one
        self.prev_cube_back = current_cube_back
        self.prev_cube_front = current_cube_front

    def _draw_connection_lines(self, front, back):
        """Draw lines connecting the front and back faces of the
cube."""
        front_vertices = [
            (front[0], front[1]),  # Top-left
            (front[0] + front[2], front[1]),  # Top-right
            (front[0] + front[2], front[1] + front[3]),  # Bottom-
right
            (front[0], front[1] + front[3]),  # Bottom-left
        ]
        back_vertices = [
            (back[0], back[1]),
            (back[0] + back[2], back[1]),
            (back[0] + back[2], back[1] + back[3]),
            (back[0], back[1] + back[3]),
        ]

        for front_point, back_point in zip(front_vertices,
back_vertices):
            pygame.draw.line(self.screen, (0, 0, 0), front_point,
back_point, 2)

    def _clear_connection_lines(self):
        """Clear lines connecting the front and back faces of the
cube."""
```

```
        if not self.prev_cube_front or not self.prev_cube_back:
            return

        front = self.prev_cube_front
        back = self.prev_cube_back

        front_vertices = [
            (front[0], front[1]),  # Top-left
            (front[0] + front[2], front[1]),  # Top-right
            (front[0] + front[2], front[1] + front[3]),  # Bottom-
right
            (front[0], front[1] + front[3]),  # Bottom-left
        ]
        back_vertices = [
            (back[0], back[1]),
            (back[0] + back[2], back[1]),
            (back[0] + back[2], back[1] + back[3]),
            (back[0], back[1] + back[3]),
        ]

        for front_point, back_point in zip(front_vertices,
back_vertices):
            pygame.draw.line(self.screen, (255, 255, 255),
front_point, back_point, 2)


    def draw(self, color):
    # Малюємо передній прямокутник
        if self.start_x is not None and self.start_y is not None
and self.end_x is not None and self.end_y is not None:
            front_coords = [self.start_x, self.start_y,
self.end_x, self.end_y]
            RectShape.detect_start(self, (front_coords[0],
front_coords[1]))
            RectShape.detect_end(self, (front_coords[2],
front_coords[3]))
            RectShape.draw(self, color, False)

    # Зміщення для заднього прямокутника
            offset = 50  # Наприклад, зсув по осі X і Y
            back_coords = [front_coords[0] + offset,
front_coords[1] + offset, front_coords[2] + offset,
front_coords[3] + offset]
            RectShape.detect_start(self, (back_coords[0],
back_coords[1]))
            RectShape.detect_end(self, (back_coords[2],
back_coords[3]))
            RectShape.draw(self, color, False)

    # З'єднуємо відповідні вершини переднього і заднього
прямокутників
            front_vertices = [
```

```
                (front_coords[0], front_coords[1]),  # Верхня ліва
                (front_coords[2], front_coords[1]),  # Верхня
права
                (front_coords[2], front_coords[3]),  # Нижня права
                (front_coords[0], front_coords[3]),  # Нижня ліва
            ]
            back_vertices = [
                (back_coords[0], back_coords[1]),
                (back_coords[2], back_coords[1]),
                (back_coords[2], back_coords[3]),
                (back_coords[0], back_coords[3]),
            ]

            for front, back in zip(front_vertices, back_vertices):
                LineShape.detect_start(self, front)
                LineShape.detect_end(self, back)
                LineShape.draw(self, color)
```

**MyTable**

```
import pygame

class MyTable:
    instance = None

    def __new__(cls):
        if MyTable.instance is None:
            MyTable.instance = super().__new__(cls)
        return MyTable.instance

    def __init__(self):
        self.screen = None
        self.acceleretor = []
        self.show_ghost_table = False
        self.ghost_table = None
        self.table_data = [["ID", "Start", "End"]]
        self.scroll_offset = 0


    def store_figure(self, element):
        self.table_data.append(element)
        print(self.table_data)

    def handle_events(self):
        if self.show_ghost_table:
            self.show_ghost_table = False
            # pygame.display.update((10, 400, 580, 190))
            self.screen.fill((255, 255, 255), self.ghost_table)
            print('action happens
<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<')
            return False
        else:
```

```
            self.show_ghost_table = True
            print('action happens
>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>')
            return True


    def add_element_in_table(self, element):
        self.table_data.append(element)

    def draw_table(self):
        if self.show_ghost_table:
            ROW_HEIGHT = 40
            VISIBLE_ROWS = 5  # Кількість видимих рядків у таблиці
            table_coord_x = 10
            table_coord_y = 400

            data = self.table_data
            scroll = self.scroll_offset
            table_width = 580
            col_widths = [int(table_width * 0.2), int(table_width
* 0.4), int(table_width * 0.4)]

            # Ініціалізація кольорів
            WHITE = (255, 255, 255)
            BLACK = (0, 0, 0)
            GRAY = (200, 200, 200)

            # Ініціалізація шрифту
            font = pygame.font.Font(None, 36)

            # Малювання таблиці
            self.ghost_table = pygame.draw.rect(self.screen,
'grey', [table_coord_x, table_coord_y, table_width, ROW_HEIGHT *
VISIBLE_ROWS])

            # Відображення видимих даних
            visible_data = data[scroll:scroll + VISIBLE_ROWS]
            for i, row in enumerate(visible_data):
                for j, cell in enumerate(row):
                    cell_x = table_coord_x + sum(col_widths[:j])
                    cell_y = table_coord_y + i * ROW_HEIGHT
                    cell_rect = pygame.Rect(cell_x, cell_y,
col_widths[j], ROW_HEIGHT)

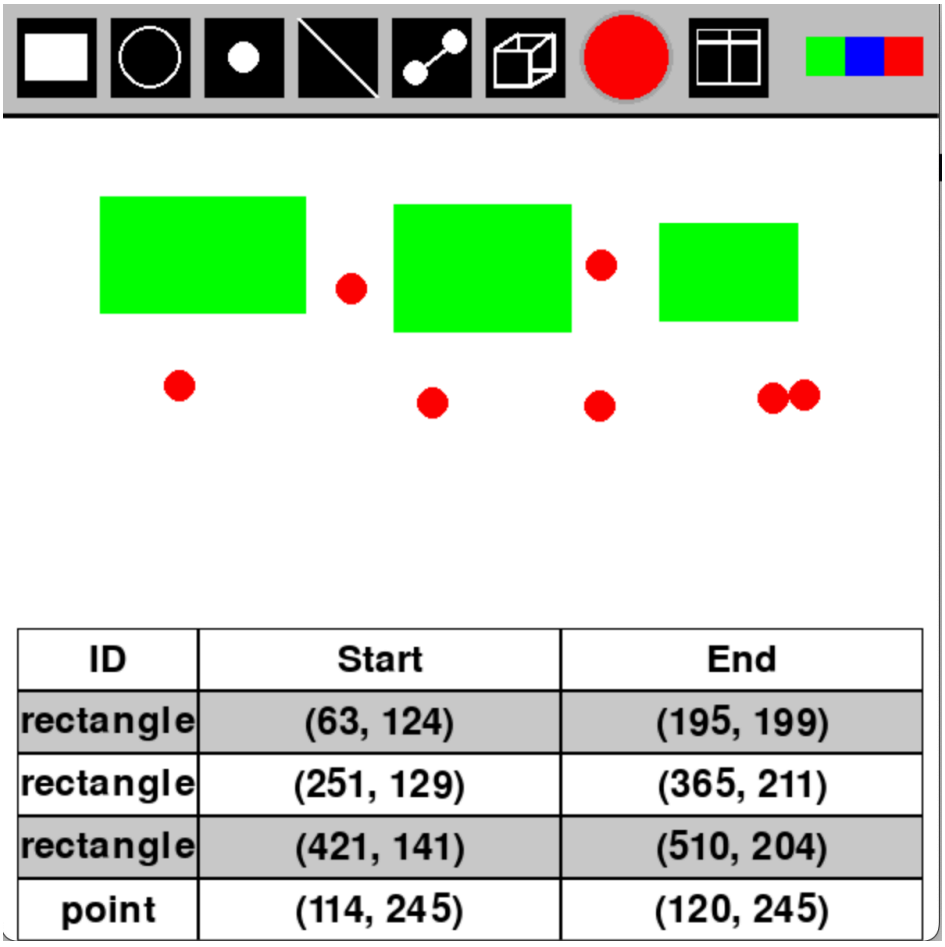
                    # Заповнення комірки
                    pygame.draw.rect(self.screen, WHITE if i % 2
== 0 else GRAY, cell_rect)
                    pygame.draw.rect(self.screen, BLACK,
cell_rect, 1)

                    # Текст у комірці
                    text = font.render(str(cell), True, BLACK)
```

```
                    text_rect =
text.get_rect(center=cell_rect.center)
                    self.screen.blit(text, text_rect)

    def handle_scroll(self, event):
        """Обробка скролу миші."""
        if self.show_ghost_table:
            # Оновлення зсуву
            self.scroll_offset -= event.y  # y: +1 (вверх), -1
(вниз)
            # Обмеження скролу в межах таблиці
            max_offset = max(0, len(self.table_data) - 5)  # 5 =
VISIBLE_ROWS
            self.scroll_offset = max(0, min(self.scroll_offset,
max_offset))
            # Оновлення відображення таблиці
            self.draw_table()
```

**Скріни проги:**



| ID | Start | End |
|---|---|---|
| rectangle | (63, 124) | (195, 199) |
| rectangle | (251, 129) | (365, 211) |
| rectangle | (421, 141) | (510, 204) |
| point | (114, 245) | (120, 245) |

## Діаграма класів програми:




## Висновок:

У цій роботі я більше дізнався про парадигму ООП, також познайомився з бібліотекою pygame ця бібліотека призначення в більшості для розробки ігор, але для застосунків такого плану теж підходить через велику кількість корисних інструментів. Також я мав практику по розбиттю задачі на підзадачі.